

CAREERFIT AI

A practical 20-point blueprint for an evidence-grounded resume and job-description matching application

PROJECT GUIDE

PDF parsing | Structured extraction | Transparent scoring | RAG evidence | Career recommendations

Build objective

Upload a resume PDF and a job-description PDF, extract structured evidence, calculate explainable fit and readiness scores, and produce grounded recommendations without claiming to reproduce a proprietary ATS.

1. Project scope

CareerFit AI compares one resume with one target job description. It is mainly a structured document-comparison product; RAG strengthens evidence retrieval and tailored guidance.

- Inputs: resume PDF and job-description PDF.
- Outputs: match score, readiness score, evidence, gaps, recommendations, and interview preparation.
- Start with text-based PDFs; add OCR later.

2. Cloud-hosted open models

Use an API provider that hosts open-weight models, so inference runs remotely. Choose a currently supported instruction model with reliable JSON or schema-constrained output.

- A single provider for chat and embeddings is simplest.
- Keep API keys only in backend environment variables.
- Pin the selected model after testing because catalogues change.

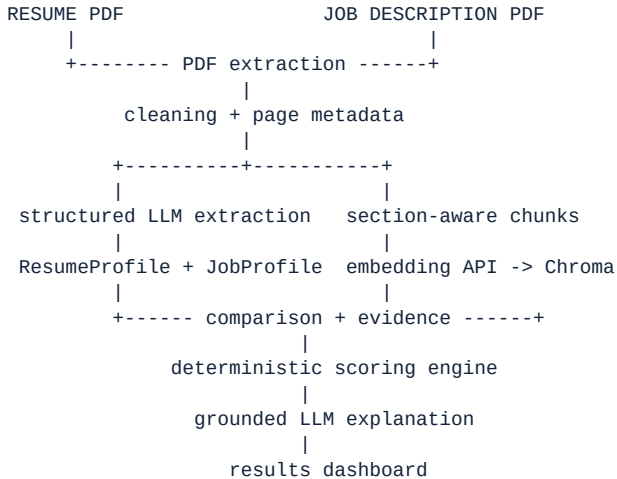
3. Embedding model

Use a hosted embedding API suited to technical vocabulary and, if needed, multilingual text. Use the same model for chunks and queries; record its name and vector dimension.

4. Local ChromaDB

Persistent local Chroma is suitable for this first single-user prototype. It stores chunk text, metadata, vectors, and indexes on disk; searching a few dozen chunks is lightweight. Filter every query by random analysis ID and document type, and provide a delete-analysis action.

Architecture and workflow



5. Four major engines

Keep responsibilities separate so each layer can be tested.

- Document engine: validate PDFs, extract text, preserve pages, and clean safely.
- Structure engine: convert raw text into validated resume and JD profiles.
- Comparison engine: exact, normalized, and semantic matching with evidence.
- Presentation engine: explain fixed scores and render the dashboard.

6. Document extraction

Use PyMuPDF page by page. Preserve page numbers and source type. Reject empty, password-protected, or image-only PDFs clearly in version one.

```
def extract_pdf_text(path):
    import fitz
    doc = fitz.open(path)
    return "\n".join(
        f"--- Page {i} ---\n{p.get_text('text')}"
        for i, p in enumerate(doc, 1)
    )
```

7. Structured information extraction

Ask the LLM for schema-constrained JSON and validate it with Pydantic. Unknown values should be null, not guesses. Resume fields include skills, roles, durations, responsibilities, tools, education, certifications, and projects. JD fields include required and preferred skills, experience, responsibilities, education, tools, and domain knowledge.

8. Transparent scoring

Do not ask an LLM to invent an ATS score. Calculate a Resume-JD Match Score with a published formula. Keep Content-based ATS Readiness separate.

Component	Weight
Required skills	40%
Experience	20%
Responsibilities	15%
Preferred skills	10%
Education	10%
Resume quality	5%

```
WEIGHTS = {
    "required_skills": .40, "experience": .20,
    "responsibilities": .15, "preferred_skills": .10,
    "education": .10, "resume_quality": .05,
}
def overall(scores):
    return round(sum(scores[k] * w for k,w in WEIGHTS.items()), 2)
```

9. Skill matching

Normalize case, punctuation, abbreviations, and known aliases before exact matching. Use semantic similarity only for remaining terms. A useful credit scheme is: exact 1.0, strong equivalent 0.9, related 0.5, missing 0.0. A related framework is not automatically the required framework.

10. Experience scoring

Compare candidate years against the stated requirement, capped at 100%. Distinguish total experience from relevant experience and label any AI-estimated duration.

```
def experience_score(candidate, required):
    if not required: return 100.0
    if candidate is None: return 0.0
    return round(min(candidate / required, 1.0) * 100, 1)
```

11. Responsibility similarity

Embed each major JD responsibility and retrieve the closest resume bullets. Display the requirement, match strength, exact resume evidence, and both page references. This is the clearest RAG demonstration.

12. Content-based ATS readiness

Measure content completeness and keyword clarity, not proprietary ATS behavior. Check contact details, summary, skills, work experience, education, headings, relevant keywords, action verbs, and quantified achievements.

13. RAG design

Store resume and JD chunks with metadata: analysis_id, document_type, section, and page. Retrieval must filter by analysis and source. Use retrieval to find evidence for a skill, locate where a JD requires a missing skill, and ground interview questions.

```
{
  "text": "Developed REST APIs using Python and Flask.",
  "metadata": {"analysis_id": "random-id",
               "document_type": "resume",
               "section": "experience", "page": 2}
}
```

14. Results dashboard

Design around decisions, not raw model output.

- Summary cards: overall match, readiness, required skills, experience.
- Skill table: requirement, status, strength, and evidence.
- Evidence viewer: requirement, candidate proof, pages, and confidence.
- Strengths and gaps without repetition.
- Recommendations separated into clearer wording, genuine learning gaps, and projects.
- Grounded technical, resume-specific, and gap-focused interview questions.

15. End-to-end data flow

Validate uploads -> extract page text -> clean conservatively -> extract and validate profiles -> create section-aware chunks -> call embedding API -> store in Chroma -> calculate deterministic scores -> compare semantically -> generate grounded explanations -> render results -> allow deletion.

16. Project structure

Use app.py plus focused modules for parsing, schemas, extraction, chunking, vector storage, skill matching, scoring, evidence retrieval, recommendations, and prompts. Keep local Chroma data and synthetic tests in separate directories.

17. Core data models

Use explicit AnalysisResult and MatchEvidence models so the UI never parses prose.

```
class AnalysisResult(BaseModel):
    analysis_id: str
    overall_match_score: float
    ats_readiness_score: float
    component_scores: dict[str, float]
    matched_skills: list[str]
    missing_skills: list[str]
    strengths: list[str]
    gaps: list[str]
    recommendations: list[str]

class MatchEvidence(BaseModel):
    requirement: str
    resume_evidence: str | None
    jd_evidence: str
    resume_page: int | None
    jd_page: int | None
    match_type: str
    confidence: float
```

18. What to defer

Do not begin with login, cloud deployment, OCR, scraping, multiple users, resume generation, fine-tuning, agents, or multiple vector databases. Add them only after the core comparison is accurate.

19. Development phases

Build vertically and verify each phase.

- Phase 1: upload and extract readable, page-aware text.
- Phase 2: produce stable validated ResumeProfile and JobProfile JSON.
- Phase 3: implement scores and verify them by hand.
- Phase 4: add chunks, embeddings, Chroma, retrieval, and citations.
- Phase 5: add grounded strengths, gaps, rewrites, learning, and interviews.
- Phase 6: evaluate 20-30 synthetic resume-JD pairs across strong, medium, and weak matches.

20. Privacy and final setup

Resumes contain personal data. Delete temporary uploads after processing, avoid logging full text, use random IDs, exclude secrets and local data from version control, and review provider retention settings before using real resumes.

Recommended first-version stack

Interface: Streamlit

Application: Python; LangChain where useful

PDF parsing: PyMuPDF

LLM: hosted open-weight instruction model via API

Embeddings: hosted embedding API

Vector storage: local persistent ChromaDB

Validation: Pydantic

Scoring: deterministic Python

Visuals: Streamlit and Plotly

Definition of done: Upload two PDFs and receive reproducible scores, matched and missing skills, page-linked evidence, honest limitations, useful recommendations, and one-click deletion.

Implementation checklist

- Use synthetic documents before real resumes.
- Validate JSON and retry only on validation failure.
- Unit-test every scoring component.
- Never give semantic similarity full credit without an equivalence rule.
- Require citations for explanation claims.
- Distinguish missing evidence from missing real-world experience.
- Document the formula and limitations in the UI and README.